

# Metodología de Sistemas II

## Trabajo Práctico 1 Factory Method

Farias, Gustavo

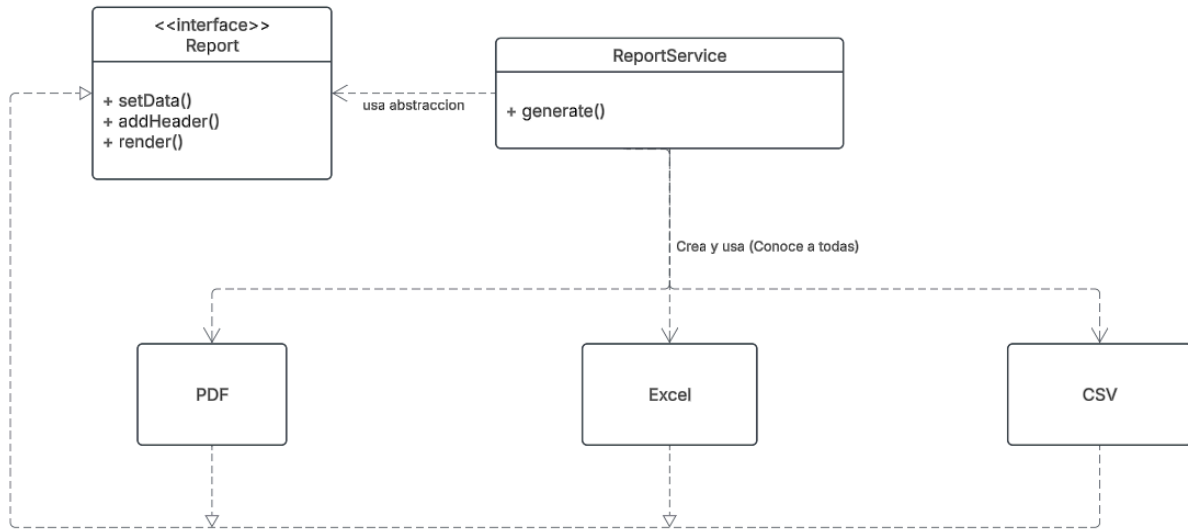
Matrícula: 101662

Comisión: M2025-13

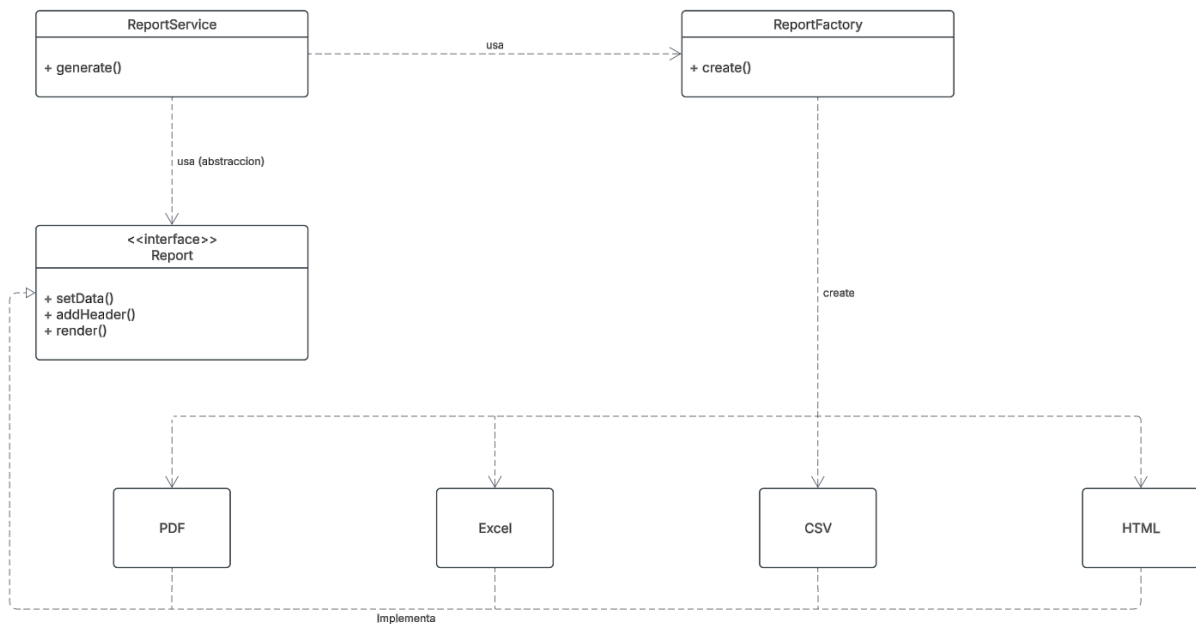
Repositorio GitHub: [Lucenear/Tecnicatura\\_Programacion\\_UTN](https://github.com/Lucenear/Tecnicatura_Programacion_UTN)

## Diagrama UML

### UML antes del acoplamiento



### UML luego del desacoplamiento



### Código Original (El problema de antes)

Este es el código problemático donde **ReportService** viola el principio open / closed y conoce a todas las clases concretas. **ReportService** importa e instancia **PDFReport**, **ExcelReport**, **CSVReport** directamente.

Report.java

```
public interface Report {
```

```

    void setData(String data);
    void addHeader(String text);
    void addFooter(String text);
    void render();
    String getOutput();
}

```

PDFReport.java, ExcelReport.java, CSVReport.java

```

public class PDFReport implements Report {
    private String data, header, footer;
    public void setData(String data) { this.data = data; }
    public void addHeader(String text) { this.header = text; }
    public void addFooter(String text) { this.footer = text; }
    public void render() { /* logica PDF */ }
    public String getOutput() { return "[PDF] " + header + " | " +
data + " | " + footer; }
}

public class ExcelReport implements Report {
    private String data, header, footer;
    public void setData(String data) { this.data = data; }
    public void addHeader(String text) { this.header = text; }
    public void addFooter(String text) { this.footer = text; }
    public void render() { /* logica Excel */ }
    public String getOutput() { return "[EXCEL] " + header + " | "
+ data + " | " + footer; }
}

public class CSVReport implements Report {
    private String data, header, footer;
    public void setData(String data) { this.data = data; }
    public void addHeader(String text) { this.header = text; }
    public void addFooter(String text) { this.footer = text; }
    public void render() { /* logica CSV */ }
    public String getOutput() { return "[CSV] " + header + " | " +
data + " | " + footer; }
}

```

ReportService.java (a resolver)

```

public class ReportService {

    // importa y conoce todas las clases concretas
    // cada nuevo formato obliga a modificar esta clase

```

```

public String generate(String data, String format_type) {
    Report report;

    // Construcción mezclada con lógica de uso
    if (format_type.equals("pdf")) {
        report = new PDFReport();
    } else if (format_type.equals("excel")) {
        report = new ExcelReport();
    } else if (format_type.equals("csv")) {
        report = new CSVReport();
    } else {
        throw new IllegalArgumentException("Formato no
soportado: " + format_type);
    }

    // la lógica de uso es igual para todos
    report.setData(data);
    report.addHeader("Reporte Mensual");
    report.addFooter("Generado el " + LocalDate.now());
    report.render();

    return report.getOutput();
}
}

```

### Código refactorizado despues:

Report.java (sin cambios)

```

public interface Report {
    void setData(String data);
    void addHeader(String text);
    void addFooter(String text);
    void render();
    String getOutput();
}

```

PDFReport.java, ExcelReport.java, CSVReport.java

```

public class PDFReport implements Report {
    private String data, header, footer;
    public void setData(String data) { this.data = data; }
    public void addHeader(String text) { this.header = text; }
    public void addFooter(String text) { this.footer = text; }
}

```

```

        public void render() { /* logica PDF */ }
        public String getOutput() { return "[PDF] " + header + " | " +
data + " | " + footer; }
    }

    public class ExcelReport implements Report {
        private String data, header, footer;
        public void setData(String data) { this.data = data; }
        public void addHeader(String text) { this.header = text; }
        public void addFooter(String text) { this.footer = text; }
        public void render() { /* logica Excel */ }
        public String getOutput() { return "[EXCEL] " + header + " | "
+ data + " | " + footer; }
    }

    public class CSVReport implements Report {
        private String data, header, footer;
        public void setData(String data) { this.data = data; }
        public void addHeader(String text) { this.header = text; }
        public void addFooter(String text) { this.footer = text; }
        public void render() { /* logica CSV */ }
        public String getOutput() { return "[CSV] " + header + " | " +
data + " | " + footer; }
    }

```

#### HTMLReport.java (nuevo)

```

    public class HTMLReport implements Report {
        private String data, header, footer;
        public void setData(String data) { this.data = data; }
        public void addHeader(String text) { this.header = text; }
        public void addFooter(String text) { this.footer = text; }
        public void render() { /* logica HTML */ }
        public String getOutput() {
            return "<html><body><h1>" + header + "</h1><p>" + data +
"</p><footer>" + footer + "</footer></body></html>";
        }
    }

```

#### ReportService.java

```

    public class ReportService {

        public String generate(String data, String format_type) {
            // El servicio no conoce ninguna clase concreta de reporte

```

```

        Report report = ReportFactory.create(format_type);

        // logica de uso
        report.setData(data);
        report.addHeader("Reporte Mensual");
        report.addFooter("Generado el " + LocalDate.now());
        report.render();

        return report.getOutput();
    }
}

```

ReportFactory.java (refactorizada)

```

public class ReportFactory {

    /*En este sistema existe un unico tipo de objeto (Report) en
    distintas variantes (PDF, Excel, CSV, HTML) que nunca se
    construyen en conjunto como familia
    */

    public static Report create(String format_type) {
        switch (format_type.toLowerCase()) {
            case "pdf": return new PDFReport();
            case "excel": return new ExcelReport();
            case "csv": return new CSVReport();
            case "html": return new HTMLReport();

            default: throw new IllegalArgumentException("Fomato no
soportado: " + format_type);
        }
    }
}

```

### Justificacion:

Se eligio Factory Method y no Abstract Factory porque el sistema crea variantes de un único Report y no existen familias de objetos relacionados que deban construirse juntos.

En cuanto a la variante elegida, decidí Static Factory porque la lógica de uso es idéntica para todos los formatos y el comportamiento del creador no varía.